

API DataJud (Elasticsearch)

José Eduardo de Souza Pimentel

Introdução

A API Pública do DataJud (CNJ) dá acesso a metadados de processos judiciais de todos os tribunais brasileiros. Diferente de uma API REST tradicional, ela é construída sobre o **Elasticsearch** — por isso toda consulta é feita via **POST**, com o corpo da requisição em um formato próprio chamado Query DSL (em JSON), e não por parâmetros simples na URL.

Estrutura da requisição (boilerplate Python)

```
import requests
import json

url = "https://api-publica.datajud.cnj.jus.br/api_publica_<alias>/_search"

payload = json.dumps({
    "query": { ... },
    "size": 10
})

headers = {
    'Authorization': 'ApiKey <Chave Pública>',
    'Content-Type': 'application/json'
}

response = requests.post(url, headers=headers, data=payload)
print(response.text)
```

Endpoints por tribunal

Padrão: `https://api-publica.datajud.cnj.jus.br/api_publica_<alias>/_search`

Tribunal	Alias
STJ	<code>stj</code>
TST	<code>tst</code>
TSE	<code>tse</code>
STM	<code>stm</code>
TRF1, TRF2, TRF3...	<code>trf1, trf2, trf3...</code>

Tribunal	Alias
TJDFT	<code>tjdft</code>
TJAM, TJSP, TJRJ...	<code>tjam</code> , <code>tjsp</code> , <code>tjrj</code> ...
TRT3, TRT15...	<code>trt3</code> , <code>trt15</code> ...

Lista completa de aliases na Datajud-Wiki (datajud-wiki.cnj.jus.br).

Cláusulas principais da Query DSL

Cláusula	Função
<code>match</code>	Busca por valor num campo (com análise textual)
<code>term</code>	Busca por valor exato, sem análise textual — melhor para códigos
<code>bool</code>	Combina múltiplas condições
<code>bool.must</code>	Todas as condições devem ser verdadeiras (E) — conta para relevância
<code>bool.filter</code>	Mesmo que <code>must</code> , mas sem cálculo de relevância — mais rápido, ideal para códigos exatos
<code>bool.should</code>	Pelo menos uma condição (OU)
<code>bool.must_not</code>	Exclui resultados (NÃO)
<code>range</code>	Filtra por intervalo (datas, números) com <code>gte/lte/gt/lt</code>
<code>nested</code>	Necessário para filtrar campos dentro de listas de objetos (ex.: <code>movimentos</code>)
<code>size</code>	Quantidade de resultados a retornar (padrão: 10)
<code>from</code>	Posição inicial, para paginação
<code>sort</code>	Define ordenação dos resultados
<code>aggs</code>	Agregações — equivalente ao <code>GROUP BY</code> do SQL

Comandos principais (exemplos prontos)

Busca simples por classe processual

```
{
  "query": {
    "match": { "classe.codigo": 1116 }
  }
}
```

Combinando condições (classe + órgão julgador)

```
{
  "query": {
    "bool": {
      "filter": [
        { "term": { "classe.codigo": 1116 } },
        { "term": { "orgaoJulgador.codigo": 13597 } }
      ]
    }
  }
}
```

Filtrar por intervalo de datas

```
{
  "query": {
    "range": {
      "dataAjuizamento": {
        "gte": "2025-01-01T00:00:00",
        "lte": "2025-12-31T23:59:59"
      }
    }
  }
}
```

Filtrar dentro de **movimentos** (campo aninhado)

```
{
  "query": {
    "nested": {
      "path": "movimentos",
      "query": {
        "range": {
          "movimentos.dataHora": { "gte": "2026-01-01T00:00:00" }
        }
      }
    }
  }
}
```

Paginação

```
{
  "query": { "match": { "classe.codigo": 1116 } },
  "size": 50,
```

```
"from": 50
}
```

Ordenação

```
{
  "query": { "match": { "classe.codigo": 1116 } },
  "sort": [ { "dataAjuizamento": "desc" } ]
}
```

Agregação por classe (contagem, sem trazer documentos)

```
{
  "size": 0,
  "aggs": {
    "por_classe": {
      "terms": { "field": "classe.codigo" }
    }
  }
}
```

Agregação por mês

```
{
  "size": 0,
  "aggs": {
    "por_mes": {
      "date_histogram": {
        "field": "dataAjuizamento",
        "calendar_interval": "month"
      }
    }
  }
}
```

Campos comuns

Campo	Conteúdo
numeroProcesso	Número do processo no formato CNJ
classe.codigo / classe.nome	Classe processual (Tabela Processual Unificada)
orgaoJulgador.codigo / orgaoJulgador.nome	Vara/câmara/turma responsável

Campo	Conteúdo
<code>assuntos.codigo / assuntos.nome</code>	Assunto(s) do processo
<code>movimentos.codigo / movimentos.nome / movimentos.dataHora</code>	Movimentações processuais
<code>dataAjuizamento</code>	Data de ajuizamento do processo
<code>grau</code>	Grau de jurisdição
<code>tribunal</code>	Sigla do tribunal
<code>formato.nome</code>	Físico ou eletrônico

Observações práticas

- O método é sempre **POST**, mesmo sendo uma consulta — corpo da requisição vai em `data/payload`, não em parâmetros de URL
- Datas retornadas costumam vir em UTC — considerar conversão de fuso horário antes de agrupar por dia/hora
- Para campos de código exato (`classe.codigo`, `orgaoJulgador.codigo`), prefira `term/filter` a `match/must` — mais rápido e mais preciso
- Paginação via `from+size` tem limite (geralmente 10.000 resultados); para extrações maiores, usar `scroll` ou `search_after`
- A API não indexa CPF/CNPJ nos campos de busca (restrição de privacidade/LGPD)